

Challenge Write-Up Template

Instructions: Copy this template for every challenge you submit. Fill in every section — a write-up that only shows the final flag without explanation will not receive full credit. Delete these italicized instruction lines before submitting.

Challenge Information

Student Name	<i>Facon, Janna Mae L.</i>
Section / Class	<i>BSIT- 3G</i>
Challenge Name	<i>Undo</i>
Category	<i>General Skills</i>
Points / Difficulty	<i>Easy</i>
Date Solved	<i>8/13/2026</i>
Flag Format Given	<i>PicoCTF{...}</i>

1. Challenge Description

Restate the challenge in your own words: what were you given, and what were you asked to find or do? Do not simply copypaste the original prompt.

The challenge provided a transformed version of a flag and asked me to reverse a series of Linux text transformations to recover the original flag. Each step gave a transformed flag together with a hint explaining what transformation had been applied. My task was to identify the correct Linux command that would reverse each transformation until the original PicoCTF flag was recovered

2. Initial Analysis / Reconnaissance

What did you check first? What did file, strings, a webpage view-source, or another first look tell you? What category did you decide this was, and why?

The first step that I did was read the challenge description carefully to understand what I needed to accomplish. I learned that the flag had been modified several times using different Linux text transformations and that I needed to reverse each

transformation step by step. The challenge also provided a hint for every step, which helped identify the appropriate Linux command. I recognized that this was a **Linux command-line and text manipulation** challenge because the main task was to use commands such as `base64`, `rev`, and `tr` to restore the original text.

3. Tools and Commands Used

List every tool/command you used, in the order you used them, and what each one told you.

Tool / Command	What it was used for	Result / Output
<code>base64 -d</code>	To decode the Base64-encoded text	Recovered the text from Step 1
<code>rev</code>	To try reading the flag normally	Returned <code>Permission denied</code>
<code>tr '-' '_'</code>	To change dashes back into underscores	Recovered the original underscore characters
<code>tr '()' '{}'</code>	To change parentheses back into curly braces	Restored the flag's curly braces
<code>tr 'A-Za-z' 'N-ZA-Mn-za-m'</code>	To reverse the ROT13 transformation	Restored the original letters

4. Solution Walkthrough

Explain your reasoning step by step — not just what you typed, but why you tried it and what you concluded from the result. Include screenshots, code snippets, or command output where useful. A reader who has not seen the challenge should be able to follow your logic and arrive at the same answer.

Step 1

The first step that I did was read the challenge description to understand the objective. I learned that I was given a transformed flag and needed to reverse several Linux text transformations to recover the original flag. The challenge provided hints for each transformation, so I used those hints to determine which command I needed at every step.

by Yahaya Meddy - picoCTF 2026

INSTANCE

Additional details appear once you launch your instance.

1 Reveal hint

The second step was to look at the first transformed flag and its hint. The hint indicated that the text had been **Base64 encoded**. To reverse this transformation, I used the `base64 -d` command. After decoding the text, the terminal confirmed that the first transformation had been successfully reversed and provided the transformed flag for the next step.

[illegible]

The third step involved reversing the order of the characters. The hint stated that the text had been reversed, so I used the `rev` command. This reversed the characters back to their previous order, and the terminal displayed **“Correct!”**, confirming that the transformation had been successfully undone.

```

You can remove it using the following command:

$ rm -rf /tmp/.challenge

Your goal: step by step, reverse the original flag.
At each step, you'll see the transformed flag and a hint.
Enter the correct Linux command to reverse the last transformation.

--- Step 1 ---
Current flag: KlgphnJchm1sDvF7qg12WwV33p5u6s18bNClxtp8R7881212
Hint: Base64 encoded the string.
Enter the Linux command to reverse it: base64 -d
Correct!

--- Step 2 ---
Current flag: j1899W0-Fm9p9mFuh0-g9g-t8Fv8v10W9w0
Hint: Reversed the text.
Enter the Linux command to reverse it: rev
Correct!

--- Step 3 ---
Current flag: 99p9W0(1r1v1F1at_g9g_9u9f0m9d9t0f_507112)
Hint: Replaced underscores with dashes.
Enter the Linux command to reverse it: tr '_' '-'
Correct!

--- Step 4 ---
Current flag: 99p9W0(1r1v1F1at_g9g_9u9f0m9d9t0f_507112)
Hint: Replaced curly braces with parentheses.
Enter the Linux command to reverse it: tr '{' '(' '}' ')'
Correct!

--- Step 5 ---
Current flag: 99p9W0(1r1v1F1at_g9g_9u9f0m9d9t0f_507112)
Hint: Applied ROT13 to letters.
Enter the Linux command to reverse it: tr 'N-Zm-n' 'A-Za-z'
Correct!

Congratulations! You've recovered the original flag:
tr 'N-Zm-n' 'A-Za-z' 'A-Za-z' 'A-Za-z'

```

Step 4

The fourth step involved fixing the characters that had been changed from underscores to dashes. The hint explained that underscores had been replaced with dashes. I reversed this transformation using `tr '-' '_'`, which changed the dashes back into underscores. The terminal again confirmed that the answer was correct.

```

You can remove it using the following command:

$ rm -rf /tmp/.challenge

Your goal: step by step, reverse the original flag.
At each step, you'll see the transformed flag and a hint.
Enter the correct Linux command to reverse the last transformation.

--- Step 1 ---
Current flag: KlgphnJchm1sDvF7qg12WwV33p5u6s18bNClxtp8R7881212
Hint: Base64 encoded the string.
Enter the Linux command to reverse it: base64 -d
Correct!

--- Step 2 ---
Current flag: j1899W0-Fm9p9mFuh0-g9g-t8Fv8v10W9w0
Hint: Reversed the text.
Enter the Linux command to reverse it: rev
Correct!

--- Step 3 ---
Current flag: 99p9W0(1r1v1F1at_g9g_9u9f0m9d9t0f_507112)
Hint: Replaced underscores with dashes.
Enter the Linux command to reverse it: tr '_' '-'
Correct!

--- Step 4 ---
Current flag: 99p9W0(1r1v1F1at_g9g_9u9f0m9d9t0f_507112)
Hint: Replaced curly braces with parentheses.
Enter the Linux command to reverse it: tr '{' '(' '}' ')'
Correct!

--- Step 5 ---
Current flag: 99p9W0(1r1v1F1at_g9g_9u9f0m9d9t0f_507112)
Hint: Applied ROT13 to letters.
Enter the Linux command to reverse it: tr 'N-Zm-n' 'A-Za-z'
Correct!

Congratulations! You've recovered the original flag:
tr 'N-Zm-n' 'A-Za-z' 'A-Za-z' 'A-Za-z'

```


6. Key Concept / What I Learned

In 2-4 sentences, name the underlying security concept or vulnerability class this challenge taught (e.g. "weak RSA exponent," "reflected XSS," "stack buffer overflow"). This is the part of the write-up most worth remembering after the flag is submitted.

*The main concept I learned from this challenge is **reversing Linux text transformations using command-line tools**. Different commands such as `base64 -d`, `rev`, and `tr` can be used to undo common text transformations and recover the original data. This challenge also showed the importance of carefully reading hints and understanding how each transformation changes the original text.*

7. References

List any external resource you consulted beyond course materials (a tool's documentation, a technique explainer, a similar public write-up). It is normal and expected to use references — just cite them.

https://www.youtube.com/watch?v=qTKdqTu7u_A
